Two-Level Autoresearch: Optimizing LLM Policy Synthesis in Sequential Social Dilemmas

Víctor Gallego
victor.gallego@komorebi.ai
Komorebi AI Technologies
Madrid, Spain

Abstract

We demonstrate an autonomous AI agent that discovers effective configurations for a complex multi-agent system—without human intervention. A *researcher agent* $\mathcal{R}$ (Claude Opus 4.6) iteratively modifies the synthesis pipeline—system prompts, feedback construction, helper libraries, iteration logic—under which a *policy-synthesizer* LLM $\mathcal{M}$ generates cooperative strategies for Sequential Social Dilemmas. The researcher reads code, edits source files, runs evaluations, and decides what to try next, following the *autoresearch* paradigm. We conduct 12 autonomous runs across two games (Cleanup and Gathering), two policy LLMs (Gemini 3.1 Pro and Claude Sonnet 4.6), and two social welfare objectives (utilitarian efficiency and Rawlsian maximin). The researcher reliably exceeds baselines: in Cleanup, efficiency improves by 66% (Gemini) and 263% (Sonnet), while dramatically tightening run-to-run spread (max/mean gap: 0.05 for Gemini and 0.02 for Sonnet, vs. baseline ranges of ~1.5). Most strikingly, maximin-optimized pipelines achieve near-perfect equality (E=0.98) with negligible efficiency loss (−1%), discovering duty rotation as the key fairness mechanism—never found under efficiency optimization. The full-pipeline approach substantially outperforms prompt-only optimization (GEPA), achieving 3× higher efficiency in Cleanup and 37% higher in Gathering, as prompt-level search cannot overcome the high variance of LLM code generation. The contrast with Gathering, where efficiency optimization alone yields E>0.94, reveals that game structure determines whether fairness requires explicit optimization: in provision dilemmas fairness needs designed mechanisms, while in restraint dilemmas it emerges as a byproduct of coordination.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; **Cooperation and coordination**; *Machine learning*.

Keywords

sequential social dilemmas, LLM policy synthesis, multi-agent systems, automated research, mechanism design

ACM Reference Format:
Víctor Gallego. 2026. Two-Level Autoresearch: Optimizing LLM Policy Synthesis in Sequential Social Dilemmas. In . ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Background

We build on the iterative LLM policy synthesis framework of Gallego [3]. An SSD is a partially observable Markov game $\mathcal{G} = \langle N, \mathcal{S}, \{A_i\}_{i=1}^N, T, \{R_i\}_{i=1}^N, \gamma \rangle$. Social outcomes are evaluated via four metrics [8]: efficiency U, equality E, sustainability S, and peace P. We additionally consider the *maximin* (Rawlsian) welfare criterion $\min_i R_i$, which maximizes the return of the worst-off agent.

We study two SSDs that represent complementary dilemma types. **Cleanup** [4] is a *public goods provision* game (N=10): a river accumulates waste while an orchard grows apples. Cleaning costs −1 per beam but enables apple regrowth for everyone (+1 per apple). The dilemma: free-ride on cleaning vs. contribute. **Gathering** [7, 8] is a *common pool resource* game (N=4): agents collect shared apples and may tag rivals to temporarily remove them. The dilemma: over-harvest and aggress vs. cooperate and share. The games differ in their cost structure—asymmetric provision vs. symmetric restraint—a distinction that drives our key findings.

A frozen LLM $\mathcal{M}$ acts as a policy synthesizer. Given a system prompt p and a feedback prompt q_k, it generates a Python policy $\pi_{k+1} = \mathcal{M}(p, q(\pi_k, \mathcal{F}_k^\ell))$, where ℓ specifies the feedback level and $\mathcal{F}_k = (\bar{r}_k, \mathbf{m}_k)$ contains the mean per-agent reward $\bar{r}_k$ and the social metrics vector $\mathbf{m}_k = (U_k, E_k, S_k, P_k)$. All N agents execute the same policy (homogeneous self-play). Gallego [3] compared two hand-designed feedback levels—sparse (reward-only) and dense (reward + social metrics)—and showed that dense feedback consistently improves cooperative outcomes. The dense feedback configuration achieves U=1.93 (mean) / 2.70 (max) with Gemini and U=0.86 / 1.56 with Sonnet on Cleanup (4 runs each).

2 Two-Level Framework

We introduce a two-level system where a *researcher agent* $\mathcal{R}$ autonomously discovers configurations that optimize the output of an inner-loop system. While we instantiate this for multi-agent policy synthesis, the architecture is general: any pipeline where an LLM generates artifacts, evaluates them, and iterates can serve as the inner loop. The key insight is that the entire inner-loop codebase—not just the feedback level—is a designable artifact that an AI agent can search over. Figure 1 illustrates the architecture.

2.1 Configuration Space

Let $\mathcal{C}$ denote the space of *pipeline configurations*. Each configuration $c \in \mathcal{C}$ specifies the full inner-loop setup:

$$c = (p,\ \ell,\ \phi,\ \mathcal{H},\ \iota,\ \nu) \tag{1}$$

where p is the system prompt, ℓ is the feedback template, ϕ is the feedback construction function, $\mathcal{H}$ is the helper function library, ι specifies the iteration logic (number of iterations K, sampling strategy, temperature schedule), and ν is the validation pipeline. Table 1 provides concrete examples.

Unpublished working draft. Not for distribution.

Conference'17, Washington, DC, USA
https://doi.org/10.1145/nnnnnnn.nnnnnnn

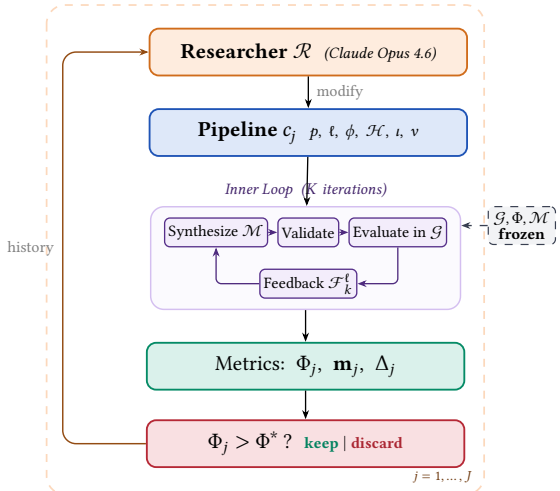

Figure 1: Two-level automated research framework (Algorithm 1). *Outer loop*: the researcher $\mathcal{R}$ modifies the pipeline configuration c_j, runs the inner loop, and keeps or discards based on Φ. *Inner loop*: the policy LLM $\mathcal{M}$ iteratively synthesizes, validates, and evaluates policies in the frozen environment $\mathcal{G}$.

Table 1: Configuration components modifiable by the researcher. The environment simulator, ground-truth evaluation, and policy LLM weights are frozen.

	Scope	Examples
p	System prompt	Hints, worked examples
ℓ	Feedback template	Metric subset, framing
ϕ	Feedback fn	Derived metrics, trends
$\mathcal{H}$	Helper library	Pathfinding, zones
ι	Iteration logic	K, eval seeds, best-of-n
ν	Validation	Safety checks, smoke tests

The sparse vs. dense comparison of [3] corresponds to a single binary choice within the ℓ component. Our framework opens the full configuration space to automated search.

2.2 Inner Loop (Policy Synthesis)

Given a configuration c, the inner loop executes K iterations of LLM policy synthesis:

$$\pi_c^* = \textsc{InnerLoop}(\mathcal{M}, \mathcal{G}, c) \tag{2}$$

Each iteration k proceeds in four stages, following [3]:

(1) **Synthesize.** The policy LLM $\mathcal{M}$ receives the system prompt p, the previous policy's source code π_{k-1}, and feedback $\mathcal{F}_{k-1}^\ell$ constructed according to (ℓ, ϕ). It generates a new Python policy function π_k that has access to full environment state and the helper library $\mathcal{H}$.

(2) **Validate.** The generated code undergoes AST-based safety checks (blocking dangerous operations such as file I/O and network access) followed by a short smoke test. Failures trigger re-generation (up to R retries), with the error message appended to the prompt.

Algorithm 1 Two-Level Automated Research

Require: Game $\mathcal{G}$, policy synthesizer $\mathcal{M}$, researcher $\mathcal{R}$, system prompt for researcher $p_{\mathcal{R}}$, initial configuration c_0, outer iterations J, ground-truth objective Φ, held-out seeds $S_{\text{held-out}}$

Ensure: Best configuration c^*, best policy π^*

```
π*_0 ← InnerLoop(M, G, c_0)
Φ_0 ← Eval(π*_0; G, S_held-out)
history ← {(c_0, Φ_0, m_0, ∅)}
for j = 1, ..., J do
    c_j ← R(p_R, code(c_{j-1}), history)
    if ¬ ValidateConfig(c_j) then retry ≤ R times
    π*_j ← InnerLoop(M, G, c_j)
    Φ_j ← Eval(π*_j; G, S_held-out)
    Δ_j ← Diff(c_{j-1}, c_j)                    // code diff
    history ← history ∪ {(c_j, Φ_j, m_j, Δ_j)}
end for
j* ← arg max_j Φ_j
return c_{j*}, π*_{j*}
```

(3) **Evaluate.** All N agents execute the same policy π_k in self-play over $|S|$ random seeds. The evaluation yields the mean per-agent reward $\bar{r}_k$ and the social metrics vector $\mathbf{m}_k = (U_k, E_k, S_k, P_k)$.

(4) **Feedback.** The feedback function ϕ constructs the prompt for the next iteration from $(\pi_k, \bar{r}_k, \mathbf{m}_k)$ according to the template ℓ. Under sparse feedback, $\mathcal{M}$ sees only the scalar reward; under dense feedback, it additionally receives all social metrics with natural-language definitions.

The inner loop output is evaluated on held-out seeds via a fixed ground-truth objective:

$$\Phi(c) = \textsc{Eval}(\pi_c^*;\ \mathcal{G},\ S_{\text{held-out}}) \tag{3}$$

where Φ maps from social outcomes to a scalar. We consider two objectives:

$$\Phi_U = U \qquad \text{(utilitarian efficiency)} \tag{4}$$

$$\Phi_{\min} = \min_i R_i \qquad \text{(Rawlsian maximin)} \tag{5}$$

2.3 Outer Loop (Automated Research)

The researcher agent $\mathcal{R}$ iteratively modifies the pipeline configuration. Following the autoresearch paradigm [6], $\mathcal{R}$ operates on the inner-loop codebase as a modifiable artifact, proposing changes, observing outcomes, and refining. The procedure is formalized in Algorithm 1.

At each outer iteration j, the researcher $\mathcal{R}$ receives:

- The **full source code** of the current configuration c_{j-1} (prompts, feedback construction, helpers, iteration logic).
- The **experiment history**: for each prior iteration, the code diff Δ_i, ground-truth score Φ_i, and social metrics vector $\mathbf{m}_i$.
- The **environment source code** (read-only), enabling the researcher to reason about game mechanics.

The researcher proposes a new configuration c_j by generating code modifications. Concretely, $\mathcal{R}$ is a coding agent (Claude Code CLI) that operates on a real software repository: it reads and edits Python source files, runs shell commands, inspects evaluation outputs, and commits changes to a dedicated git branch—the same

Unpublished working draft. Not for distribution.

Table 2: Structural analogy between autoresearch [6] for LLM pretraining and our framework for multi-agent policy synthesis.

	Autoresearch	This work
Modifiable artifact	`train.py` (model + optimizer)	Inner-loop codebase $(p, \ell, \phi, \mathcal{H}, \iota, \nu)$
Frozen infrastructure	`prepare.py` (data, tokenizer, eval)	Environment $\mathcal{G}$, eval Φ, LLM $\mathcal{M}$
Metric	val_bpb ↓	Φ (social welfare) ↑
Budget	5 min wall clock	K iters × $\lvert S\rvert$ seeds
Agent	AI coding agent	Researcher $\mathcal{R}$

workflow a human researcher would follow. This mirrors the autoresearch setup, where an AI coding agent modifies `train.py` and observes validation loss; here, the researcher modifies the inner-loop codebase and observes social welfare.

2.4 Structural Analogy

Table 2 formalizes the correspondence between autoresearch for LLM pretraining and our two-level system.

The key structural property shared by both systems: the agent modifies *how learning happens* (the pipeline/training code), not the learned artifact directly (the policy/model weights). The output of each outer iteration is a *reusable configuration*—transferable across policy LLMs, games, and random seeds—rather than a single policy.

2.5 Design Principles

Separation of concerns. The researcher $\mathcal{R}$ and the synthesizer $\mathcal{M}$ are distinct LLM calls with distinct roles. $\mathcal{R}$ reasons about *what information and tools help $\mathcal{M}$ write better policies*; $\mathcal{M}$ reasons about *what policy to write given its current context.* Using different underlying models ($\mathcal{R} \neq \mathcal{M}$) avoids conflation and enables cross-model experiments.

Read-only environment. The researcher can read the environment source code to understand game mechanics but cannot modify the simulator. This prevents the trivial solution of making the game easier and cleanly separates mechanism design from environment design.

Fixed compute budget. Each inner loop runs for exactly K iterations with $|S|$ seeds. The researcher cannot inflate performance by running more iterations, but can change what happens *within* those K iterations (e.g., best-of-n sampling, varying the prompt structure mid-run).

Diff-based history. The researcher sees code diffs Δ_j from previous experiments alongside their outcomes. This enables causal reasoning about which modifications drove which improvements, mirroring autoresearch's experiment logs.

3 Connection to Automated Mechanism Design

The two-level structure admits a mechanism design interpretation. The researcher $\mathcal{R}$ acts as a *mechanism designer*: it controls the information structure (what metrics to reveal, how to frame them), the action space (what helper functions are available), and the incentive structure (how feedback is presented) under which the policy synthesizer $\mathcal{M}$ operates. The synthesizer acts as the *agent* within the designed mechanism.

This connects to the automated mechanism design literature [2], where a principal designs rules to induce desired behavior from self-interested agents. In our setting:

- The **principal** is the researcher $\mathcal{R}$, optimizing the ground-truth objective Φ.
- The **agent** is the synthesizer $\mathcal{M}$, optimizing per-agent reward as instructed.
- The **mechanism** is the configuration c: prompts, feedback, helpers, iteration logic.
- The **outcome** is the social welfare of the resulting multi-agent policy π_c^*.

A key distinction from classical mechanism design: $\mathcal{M}$ is not *strategically* self-interested—it follows instructions—but it has *bounded rationality* in the sense that its ability to synthesize effective policies depends on the information and tools provided. The researcher's task is thus closer to *information design* [5]: choosing what to reveal to help $\mathcal{M}$ navigate the cooperation–defection tension. Our experiments (Section 4) show that the researcher designs qualitatively different information structures depending on the welfare objective Φ, supporting this interpretation empirically.

4 Experiments

4.1 Setup

We conduct 12 autonomous researcher runs across a factorial design. The researcher agent $\mathcal{R}$ is Claude Opus 4.6, invoked via the Claude Code CLI as a coding agent. Each run operates on a dedicated git branch of a real Python codebase: $\mathcal{R}$ edits source files in `pipeline/`, executes evaluation scripts, reads metric outputs, and iterates—without human intervention.

Design. For Cleanup: 2 policy LLMs × 2 objectives × 2 replications = 8 runs. For Gathering: 2 policy LLMs × 1 objective × 2 replications = 4 runs. Maximin runs are unnecessary for Gathering because efficiency optimization alone achieves equality $E > 0.94$ (the game's symmetric cost structure makes fairness a free byproduct of coordination).

Models. Policy synthesizer $\mathcal{M}$: Gemini 3.1 Pro (Google) or Claude Sonnet 4.6 (Anthropic). Both use extended thinking. These are the same models evaluated in [3]. We additionally test with Gemma 4 26B-A4B-IT (Google), a smaller open-weight model, to probe the framework's behavior when the policy synthesizer has substantially lower capability (Appendix C).

Baselines. The hand-designed dense feedback (reward + social metrics) configuration from [3] serves as the initial pipeline c_0 for all runs. On Cleanup (N=10), this baseline achieves U=1.93/2.70 (mean/max) with Gemini and U=0.86/1.56 with Sonnet across 4

Unpublished working draft. Not for distribution.

Table 3: Results. Mean/max across 2 runs per condition (4 for Cleanup baselines). Baseline: unmodified pipeline with dense feedback from [3]. GEPA: automated prompt optimization [1] with matched compute budget.

Policy LLM	Target	U	E	$\min_i R_i$
Cleanup (N=10) — Baseline				
Gemini 3.1 Pro	—	1.93/2.70	0.17/0.62	−159/−84
Sonnet 4.6	—	0.86/1.56	−0.02/0.25	−151/−59
Cleanup (N=10) — GEPA [1]				
Gemini 3.1 Pro	Φ_U	1.60/2.16	−0.01/0.62	—
	$\Phi_{\min}$	1.76/2.76	0.90/0.96	143/245
Cleanup (N=10) — Autoresearch				
Gemini 3.1 Pro	Φ_U	**3.20**/3.25	0.55/0.61	−211/−182
	$\Phi_{\min}$	3.16/3.19	**0.98**/0.98	**290**/296
Sonnet 4.6	Φ_U	**3.12**/3.14	0.66/0.70	−196/−146
	$\Phi_{\min}$	2.57/2.93	**0.91**/0.97	**179**/200
Gathering (N=4) — Baseline				
Gemini 3.1 Pro	—	2.04/2.42	0.90/0.98	412/571
Sonnet 4.6	—	0.03/0.03	0.54/0.54	0/0
Gathering (N=4) — GEPA [1]				
Gemini 3.1 Pro	Φ_U	2.08/2.35	0.94/0.96	—
Sonnet 4.6	Φ_U	1.23/1.27	0.68/0.68	—
Gathering (N=4) — Autoresearch				
Gemini 3.1 Pro	Φ_U	2.49/2.51	0.98/0.98	582/582
Sonnet 4.6	Φ_U	2.52/2.52	0.96/0.98	576/597

runs each. We additionally compare against GEPA [1], an automated prompt optimization method that iteratively refines the system prompt via LLM reflection. GEPA optimizes only the system prompt p, whereas our researcher modifies the full pipeline $(p, \ell, \phi, \mathcal{H}, \iota, \nu)$. We give GEPA a matched compute budget: the same number of optimization steps as outer iterations used by our method (each GEPA step costs ~3 policy evaluations, comparable to our inner loop's K+1=4).

Inner loop parameters. K=3 synthesis iterations (or K=2 if the researcher elects), evaluated over $|S|$=5–12 random seeds (researcher-configurable). The researcher ran J=3–17 outer iterations per run (mean: 8.3), with a keep rate of 18%–100% (mean: 63%).

4.2 Results

Table 3 presents the main results, comparing autoresearch against the hand-designed baseline of [3] and GEPA prompt optimization [1].

Finding 1: The researcher reliably improves over hand-designed baselines and outperforms prompt-only optimization. All 12 runs show substantial improvement regardless of starting point (Figure 2). In Cleanup, the researcher exceeds the unmodified pipeline baseline by 66% with Gemini (U: 1.93 → 3.20) and 263% with Sonnet (U: 0.86 → 3.12). Strikingly, the Sonnet improvement nearly closes the gap with Gemini (U=3.12 vs. 3.20), despite a 2× baseline disadvantage (0.86 vs. 1.93). Final efficiency values cluster tightly around $U \approx$ 3.1–3.2 despite baselines varying from 0.29 to 2.70 across individual runs, suggesting the researcher reliably discovers the performance ceiling of each policy LLM. Beyond improving the mean, the researcher yields remarkably consistent results: under efficiency optimization, Gemini's runs cluster within U=3.20–3.25 (gap 0.05) despite baselines spanning 1.15–2.70, and Sonnet's

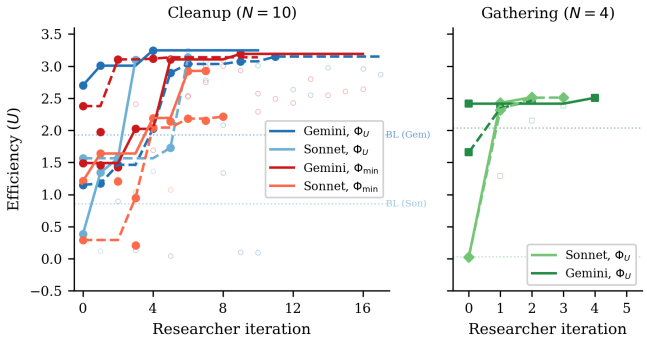

Figure 2: Efficiency (U) across researcher iterations for all 12 runs. *Left*: Cleanup (N=10) with 8 runs across 2 LLMs × 2 objectives (solid lines = kept experiments, open circles = discarded). Dashed horizontal line: hand-designed baseline from [3]. All runs converge to $U \approx$ 3.1–3.2 despite diverse starting points. *Right*: Gathering (N=4) with 4 runs (2 per LLM). All converge to $U \approx$ 2.5 within 2–4 iterations despite baselines ranging from 0.03 to 2.42.

within 3.12–3.14 (gap 0.02) from baselines of 0.38–1.56. In Gathering, all four runs converge to U=2.47–2.52 from baselines ranging 0.03–2.42. The researcher discovers pipeline modifications—structured helpers, strategic hints, tailored feedback—that constrain the policy LLM toward consistently good solutions, reducing dependence on inner-loop stochasticity.

Compared to GEPA (Table 3), autoresearch achieves 2× higher efficiency under Φ_U in Cleanup (U=3.20 vs. 1.60), 2× higher maximin under $\Phi_{\min}$ (290 vs. 143), and 20% higher efficiency in Gathering (U=2.49 vs. 2.08). Critically, GEPA shows far wider spread across runs (Cleanup Φ_U: max 2.16 vs. mean 1.60, compared to autoresearch's 3.25 vs. 3.20; $\Phi_{\min}$ maximin: 245/41 vs. autoresearch's 296/284). This highlights the advantage of modifying the full pipeline—helpers, feedback, and iteration logic provide structured scaffolding that reduces run-to-run variance, whereas prompt-only optimization cannot address the underlying stochasticity of code generation.

Finding 2: No efficiency–fairness tradeoff in Cleanup (Gemini). Maximin-optimized Gemini pipelines sacrifice only 1% efficiency (U: 3.16 vs. 3.20) while achieving near-perfect equality (E: 0.98 vs. 0.55) and transforming maximin from deeply negative baselines (−99 to −84) to $\min_i R_i$ = 290. The researcher discovers that *fair duty rotation*—time-based cycling of cleaning responsibilities using `agent_id` and `env._step_count`—simultaneously improves worst-off welfare *and* collective output. Because cleaning is a public good, distributing the cleaning cost fairly ensures enough cleaners to sustain apple production.

For Sonnet, there is a moderate tradeoff: efficiency drops from 3.12 to 2.57 under maximin optimization. The gap reflects Sonnet's harder time implementing complex coordination mechanisms (role rotation, zone assignment) from strategic hints alone.

Finding 3: Game structure determines whether fairness requires explicit optimization. In Cleanup, where cleaning costs are borne

Unpublished working draft. Not for distribution.

asymmetrically (cleaners pay −1, free-riders collect apples), baseline equality ranges from E=0.04 to 0.62, and maximin optimization is required to reach $E > 0.9$. In Gathering, where all agents face a symmetric landscape, efficiency optimization alone achieves $E > 0.94$ across all 4 runs—no separate maximin runs are needed.

This generalizes: in *provision* dilemmas with asymmetric costs, fairness requires designed mechanisms (role rotation, duty sharing); in *restraint* dilemmas with symmetric costs, fairness emerges as a free byproduct of efficient coordination. The researcher independently discovers this—it creates role differentiation pipelines only for Cleanup, and pure spatial-coordination pipelines for Gathering.

Finding 4: Convergent discovery of qualitatively different strategies per objective. Despite fully independent runs, the researcher converges on the same core strategies within each condition (Table 4, Appendix A). In Cleanup, waste-counting helpers and spatial zone partitioning appear across all 8 runs. The key qualitative difference: *role rotation*—time-based cycling of cleaning duty—appears in 4/4 maximin runs but 0/4 efficiency runs. Under efficiency optimization, the researcher assigns static cleaning roles (some agents always clean), achieving high collective output at the cost of equality. Under maximin optimization, it discovers rotation as a fairness mechanism.

In Gathering, the researcher discovers BFS-based Voronoi territory partitioning and respawn-timer awareness, with no role differentiation needed—the entire optimization is spatial (who collects which apples) and temporal (respawn-aware positioning).

Common failure modes. Several patterns led to discarded experiments across all runs: (1) *over-prescription*—adding too many strategic hints confuses the policy LLM, causing generation failures; (2) *iteration regression*—K=4 or K=5 inner iterations often cause catastrophic regression as the LLM "over-refines" a working policy; (3) *feedback overload*—dense per-agent diagnostics cause over-correction rather than gradual improvement; (4) *variance exposure*—identical configurations can yield U=3.2 then U=0.09 on different seeds, requiring multi-seed evaluation for stable signals.

5 Discussion and Conclusion

We have presented a two-level framework where an AI coding agent autonomously discovers pipeline configurations that improve the output of an inner-loop LLM system. Applied to multi-agent policy synthesis in social dilemmas, the researcher agent reliably exceeds hand-designed baselines across 12 independent runs, and converges on qualitatively similar strategies within each game–objective condition. The framework requires no domain-specific scaffolding: the agent operates on a standard software repository using the same tools (file editing, shell commands, git) available to a human researcher.

Mechanism design in action. Our results empirically support the mechanism design interpretation of Section 3. The researcher acts as an information designer: under Φ_U, it reveals efficiency-oriented information (waste counts, zone assignments) that guides $\mathcal{M}$ toward productive but potentially unequal role allocation. Under $\Phi_{\min}$, it additionally reveals fairness-oriented structure (rotation schedules, per-agent equity feedback) that guides $\mathcal{M}$ toward egalitarian coordination. The striking absence of a fairness–efficiency tradeoff with Gemini suggests that in public goods provision, the mechanism design problem has a cooperative optimum where both objectives align—the researcher discovers this independently.

Two types of dilemma. The Cleanup–Gathering contrast reveals a structural insight about social dilemmas: *provision* dilemmas (asymmetric costs, one agent's contribution benefits all) require explicit fairness mechanisms, while *restraint* dilemmas (symmetric costs, each agent faces the same temptation) achieve fairness naturally when coordination improves. This distinction, well-known in the common-pool resource literature, is rediscovered here by an autonomous AI agent—suggesting it is a robust structural feature of these environments.

Human oversight and audit. Our system is fully autonomous by design, but its architecture offers natural affordances for human-in-the-loop oversight. Because the researcher $\mathcal{R}$ operates on a standard git repository—committing each pipeline modification to a dedicated branch—a domain expert can audit the full history of code diffs Δ_j alongside their metric outcomes $(\Phi_j, \mathbf{m}_j)$, exactly as they would review a colleague's pull requests. The keep/discard decision (Algorithm 1, line 11) is a lightweight intervention point: a human could override the accept threshold, veto modifications that introduce undesirable mechanisms (e.g., exploitative role assignments), or steer the researcher toward under-explored regions of the configuration space by editing the experiment history. More broadly, the separation between the researcher $\mathcal{R}$ and the frozen evaluation Φ creates a *delegation boundary*: the human defines *what* to optimize (the welfare objective), while the agent decides *how*. This mirrors the expert-in-the-loop design patterns identified in deployment-oriented work on AI agents for discovery, where trust calibration depends on the legibility of the agent's decision trail rather than on real-time supervision.

Limitations. Our experiments use only 2 replications per condition, yielding high uncertainty on some estimates (e.g., Sonnet $\Phi_{\min}$: U=2.57/2.93). The two SSDs are small gridworld environments; scaling to larger, more complex games remains open. The researcher runs are not fully controlled—the number of outer iterations varies from 3 to 17 (mean 8.3)—making direct comparison across conditions approximate. The GEPA comparison uses a single run per condition; additional replications would strengthen confidence in the comparison.

Future work. Five directions emerge: (1) *new domains*—apply the two-level framework to other LLM-driven pipelines (code optimization, scientific experiment design, infrastructure tuning), where the same architecture applies with a different inner loop and objective Φ; (2) *ablation*—freeze all configuration components except one to isolate the relative importance of prompt, feedback, helpers, and iteration logic; (3) *adversarial objectives*—set $\Phi = R_0$ (agent 0's reward only) to test whether the researcher discovers exploitative pipeline configurations, connecting to the reward hacking analysis of [3]; (4) *cross-game transfer*—run the best Cleanup configuration on Gathering and vice versa, testing game-generality; (5) *heterogeneous policies*—extend to settings where agents run different code, enabling richer role specialization. Code is available at https://github.com/vicgalle/llm-policies-social-dilemmas.

Unpublished working draft. Not for distribution.

References

[1] Lakshya A Agrawal et al. 2026. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. *arXiv preprint arXiv:2507.19457* (2026).

[2] Vincent Conitzer and Tuomas Sandholm. 2002. Complexity of Mechanism Design. In *Proc. 18th Conference on Uncertainty in Artificial Intelligence*. 103–110.

[3] Víctor Gallego. 2026. Cooperation and Exploitation in LLM Policy Synthesis for Sequential Social Dilemmas. In *Proc. Adaptive and Learning Agents Workshop (ALA 2026)*.

[4] Edward Hughes, Joel Z Leibo, Matthew Phillips, Karl Tuyls, Edgar A Dueñez-Guzmán, Antonio García Castañeda, Iain Dunning, Tina Zhu, Kevin R McKee, R. Koster, Heather Roff, and Thore Graepel. 2018. Inequity aversion improves cooperation in intertemporal social dilemmas. In *Neural Information Processing Systems*.

[5] Emir Kamenica and Matthew Gentzkow. 2011. Bayesian Persuasion. *American Economic Review* 101, 6 (2011), 2590–2615.

[6] Andrej Karpathy. 2026. autoresearch: AI agents running research on single-GPU nanochat training automatically. https://github.com/karpathy/autoresearch.

[7] Joel Z Leibo, Vinicius Zambaldi, Marc Lanctot, Janusz Janssen, and Thore Graepel. 2017. Multi-agent reinforcement learning in sequential social dilemmas. In *Proc. 16th Conference on Autonomous Agents and MultiAgent Systems*. 464–473.

[8] Julien Perolat, Joel Z Leibo, Vinicius Zambaldi, Charles Beattie, Karl Tuyls, and Thore Graepel. 2017. A multi-agent reinforcement learning model of common-pool resource appropriation. In *Advances in Neural Information Processing Systems*, Vol. 30.

A Additional Results

Table 4: Strategies discovered by the researcher in Cleanup across 4 efficiency-optimized and 4 maximin-optimized independent runs.

Strategy	Φ_U (4 runs)	$\Phi_{\min}$ (4 runs)
Waste-counting helpers	4/4	4/4
Zone/lane partitioning	3/4	4/4
Anti-regression feedback	3/4	2/4
Worked policy examples	2/4	3/4
Cleaning cost economics	2/4	3/4
Time-based role rotation	**0/4**	**4/4**

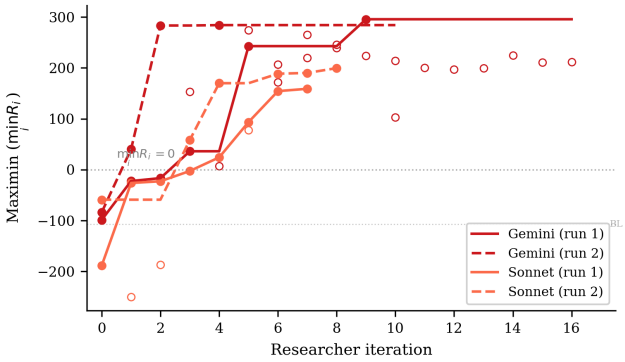

Figure 3: Maximin ($\min_i R_i$) across researcher iterations for the 4 Cleanup maximin-optimized runs. All runs transform deeply negative baselines (worst-off agents losing reward) into substantially positive values. Gemini reaches ~290 while Sonnet reaches ~160–200. The dashed line marks $\min_i R_i = 0$ (no agent loses reward overall).

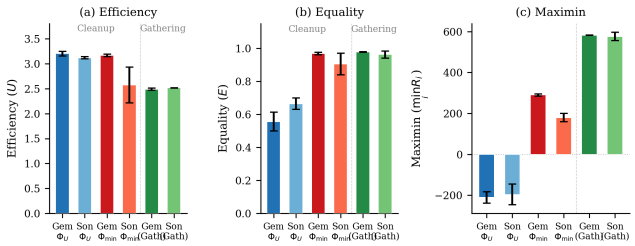

Figure 4: Final metrics across all conditions. (a) Efficiency: all conditions converge to $U \approx 2.5$–3.2. (b) Equality: maximin-optimized Cleanup runs achieve $E \approx 1.0$, while efficiency-optimized runs show $E \approx 0.5$–0.7; Gathering achieves high equality regardless. (c) Maximin: the sharpest contrast—efficiency optimization leaves the worst-off agent deeply negative ($\min_i R_i \approx -200$), while maximin optimization transforms it to +290 (Gemini) or +179 (Sonnet). Gathering achieves high maximin (~580) under efficiency optimization alone. Error bars show s.d. across runs.

B Compute Requirements

Table 5 reports wall-clock time for all 12 autonomous runs and the 6 GEPA comparison runs. *Inner loop* measures total time spent on policy LLM generation and environment simulation across all evaluations; *total* (estimated from run-directory timestamps) additionally includes the researcher agent's analysis, code editing, and decision-making between evaluations.

Table 5: Wall-clock time per autonomous run. Evals: total inner loop evaluations including initial baseline. Per eval: mean wall-clock per single evaluation. Total: end-to-end including researcher overhead.

Policy LLM	Φ	Evals	Inner loop (h)	Per eval (min)	Total (h)
Cleanup (N=10)					
Gemini	Φ_U	11	2.6	14	3.7
Gemini	Φ_U	18	4.1	14	6.4
Sonnet	Φ_U	4	2.1	32	6.1
Sonnet	Φ_U	7	5.0	43	6.1
Gemini	$\Phi_{\min}$	17	3.0	11	4.3
Gemini	$\Phi_{\min}$	11	3.6	20	5.0
Sonnet	$\Phi_{\min}$	8	4.5	33	8.8
Sonnet	$\Phi_{\min}$	9	5.3	36	13.2
Gathering (N=4)					
Sonnet	Φ_U	3	1.7	33	2.2
Gemini	Φ_U	5	1.4	17	1.9
Gemini	Φ_U	3	0.9	19	1.1
Sonnet	Φ_U	4	2.3	35	3.4
All 12 runs		100	36.6	22	62.2
GEPA (6 runs)		6	5.0	50	–

Cost breakdown. Each inner loop evaluation involves K=2–3 policy LLM generation calls (each ~2k input tokens, ~1.5k output tokens) followed by multi-seed simulation (5–12 seeds, ~15–30s total). Policy LLM generation dominates inner loop time (86–97%), with Sonnet evaluations taking ~2× longer than Gemini due

Unpublished working draft. Not for distribution.

to extended thinking. The researcher agent (Claude Opus 4.6, running via Claude Code CLI) accounts for 41% of total wall-clock time (25.6h of the 62.2h total across all 12 runs), spent reading results, editing pipeline source files, and planning modifications. In monetary terms, the researcher agent is the dominant cost: each outer iteration consumes ∼50–100k context tokens in the Opus session, whereas each inner loop evaluation uses only ∼5–10k policy LLM tokens. The 6 GEPA comparison runs required 5.0h of compute with no researcher agent, operating at lower cost per run but achieving substantially lower performance (Table 3).

C Smaller Open-Weight Model: Gemma 4 26B

We test the framework with Gemma 4 26B-A4B-IT (Google), a 26B-parameter open-weight model substantially smaller than the frontier models in the main experiments. We run three experiments—two on Cleanup (N=10) optimizing efficiency and maximin respectively, and one on Gathering (N=4) optimizing efficiency—all with Opus 4.6 as the researcher $\mathcal{R}$.

Setup. In all three experiments, Gemma starts from complete failure: the baseline pipeline produces U=−10.0 on Cleanup (agents CLEAN-spam without collecting apples) and U=0.0 on Gathering (broken BFS calls), compared to U=1.93/0.86 (Gemini/Sonnet on Cleanup) and U=2.04/0.03 (Gemini/Sonnet on Gathering). The researcher ran J=10–18 outer iterations per condition.

Results. Table 6 presents the best configurations discovered. Under efficiency optimization, the researcher achieves U=0.87—a dramatic recovery from the broken baseline but far below frontier models (U≈3.2). The researcher compensates for the model's limited capability by reducing inner-loop iterations to K=1 (avoiding the regression that plagued K≥2 runs) and providing highly structured worked examples with explicit cleaning-role assignment.

Table 6: Autoresearch with Gemma 4 26B-A4B-IT. Single run per condition. Baseline: dense feedback pipeline from [3].

Game	Target	U	E	$\min_i R_i$	Keep rate
Cleanup	Baseline	−10.0	1.00†	−1000	—
	Φ_U	0.87	−1.11	−371	6/19
	$\Phi_{\min}$	**1.71**	**0.94**	**137**	9/17
Gathering	Baseline	0.0	1.00†	0	—
	Φ_U	**2.44**	**0.98**	**580**	4/11

†Equality is trivially 1.0 because all agents receive near-zero reward.

Maximin optimization rescues efficiency. Strikingly, under maximin optimization the researcher achieves *higher* efficiency (U=1.71) than under direct efficiency optimization (U=0.87), alongside strong equality (E=0.94) and positive worst-off welfare ($\min_i R_i$=137). This reversal—absent in frontier models, where both objectives yield U≈3.1–3.2—occurs because the maximin objective forces the researcher toward coordination mechanisms (rotating cleaning duties, index-based apple assignment) that simultaneously improve collective output. Under efficiency-only optimization, the researcher converges on a local optimum—static role assignment with 25% dedicated cleaners—that a 26B model can implement but that caps performance well below the frontier.

This suggests that *for models below a capability threshold, fairness objectives may serve as better optimization targets for overall social welfare than direct efficiency maximization.* The structured coordination enforced by the maximin objective provides a scaffold that compensates for the weaker model's difficulty implementing complex strategies from strategic hints alone.

Gathering: full recovery on a simpler game. On Gathering (N=4), the researcher brings Gemma from U=0.0 to U=2.44 with E=0.98 and $\min_i R_i$=580, after 10 outer iterations (4 kept). These values nearly match frontier models (Gemini: U=2.49, Sonnet: U=2.52; Table 3). The researcher discovers the same Voronoi partitioning and respawn-aware camping strategies found in the main experiments, and increases inner-loop iterations to K=5 to allow sufficient refinement. The contrast with Cleanup—where Gemma peaks at U=0.87/1.71 versus frontier U≈3.2—suggests that the researcher can fully compensate for model weakness on simpler coordination tasks (4 agents, symmetric costs) but not on harder provision dilemmas (10 agents, asymmetric costs).

Unpublished working draft. Not for distribution.